Database Testing Documentation: MySQL

Project Description

The employee_management is a database project used to manage and to study data integrity, consistence, accuracy in MySQL. This project involves developing stable and efficient database structure based upon important relationships between employees and department to guarantee correct data handling as well as compliance with business rules. The project also focuses on establishing sound mechanisms for calling into question the validity of stored data information and its conformity to prior set standard.

Objectives:

1. Design a relational database schema to manage employee and department data.
2. Populate the database with sample data to simulate real-world scenarios.
3. Execute SQL-based test cases to verify data integrity, consistency, and accuracy.

Step 1: Define the Database Schema

The database employee_management includes two tables:

1. **Departments**: Stores department details.
2. **Employees**: Stores employee details with a reference to the department.

Schema Details:

Departments Table

Column Name	Data Type	Constraints
department_id	INT	Primary Key, Auto Increment
department_name	VARCHAR(50)	NOT NULL

Employees Table

Column Name	Data Type	Constraints
employee_id	INT	Primary Key, Auto Increment
first_name	VARCHAR(50)	-
last_name	VARCHAR(50)	-
department_id	INT	Foreign Key (Departments)
salary	DECIMAL(10, 2)	-

Step 2: Create the Database and Tables

SQL Script:

```
-- Create the database
CREATE DATABASE employee_management;

-- Use the database
USE employee_management;

-- Create the Departments table
CREATE TABLE Departments (
    department_id INT AUTO_INCREMENT PRIMARY KEY,
    department_name VARCHAR(50) NOT NULL
);

-- Create the Employees table
CREATE TABLE Employees (
    employee_id INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    department_id INT,
    salary DECIMAL(10, 2),
    FOREIGN KEY (department_id) REFERENCES Departments(department_id)
);
```

Step 3: Populate the Tables with Sample Data

SQL Script:

```
-- Insert sample data into Departments
INSERT INTO Departments (department_name)
VALUES ('Human Resources'), ('Engineering'), ('Finance'), ('Sales');

-- Insert sample data into Employees
INSERT INTO Employees (first_name, last_name, department_id, salary)
VALUES
('Alice', 'Smith', 1, 60000.00),
('Bob', 'Johnson', 2, 80000.00),
('Charlie', 'Williams', 2, 90000.00),
('Diana', 'Brown', 3, 75000.00),
('Eve', 'Davis', 4, 50000.00);
```

Step 4: Perform Database Testing

Test Cases and SQL Queries:

Test Case 1: Verify that all departments have unique IDs

```
SELECT department_id, COUNT(*)
FROM Departments
GROUP BY department_id
HAVING COUNT(*) > 1;
```

- **Expected Result**: No results returned.

Test Case 2: Check that every employee belongs to a valid department

```
SELECT e.employee_id, e.first_name, e.department_id
FROM Employees e
LEFT JOIN Departments d ON e.department_id = d.department_id
WHERE d.department_id IS NULL;
```

- **Expected Result**: No results returned.

Test Case 3: Validate that employee salaries are within a specific range (e.g., > $30,000 and < $150,000)

```
SELECT employee_id, first_name, salary
FROM Employees
WHERE salary NOT BETWEEN 30000 AND 150000;
```

- **Expected Result**: No results returned.

Test Case 4: Ensure no duplicate employee names

```
SELECT first_name, last_name, COUNT(*)
FROM Employees
GROUP BY first_name, last_name
HAVING COUNT(*) > 1;
```

- **Expected Result**: No results returned.

Test Case 5: Verify total salary budget per department

```
SELECT d.department_name, SUM(e.salary) AS total_salary
FROM Employees e
JOIN Departments d ON e.department_id = d.department_id
GROUP BY d.department_name;
```

- **Expected Result**: Correct salary sums for each department.

department_name	total_salary
Human Resources	60000.00
Engineering	170000.00
Finance	75000.00
Sales	50000.00

Testing Results

Results Summary:

1. **Test Case 1**: All department IDs are unique.
2. **Test Case 2**: All employees belong to valid departments.
3. **Test Case 3**: All salaries are within the defined range.
4. **Test Case 4**: No duplicate employee names exist.
5. **Test Case 5**: Salary budgets per department were validated and found correct.

Conclusion

The database employee_management was successfully tested for data integrity, consistency, and accuracy. All test cases passed without issues, ensuring the database is reliable for use in production.